Rupeeflow — Infrastructure Scale Guide

Fix Servers Now · Build for 3 Crore/Day

Date: March 2026 **Current:** Single EC2 m6a.large · RDS Multi-AZ · Redis on EC2 · 64% success rate **Target:** 3 Crore transactions/day · 99.9% success rate **Classification:** Internal Engineering — Confidential

---

STOP — Fix These Right Now (Before Reading Anything Else)

Your `ecosystem.config.js` has two critical bugs that are **directly causing your 64% success rate today.**

Bug 1 — Wrong PM2 Instance Count

```
// YOUR CURRENT CONFIG (WRONG)
instances: 6,  // comment says "8-core CPU"
               // m6a.large has 2 vCPU — NOT 8
               // 6 processes on 2 cores = 3 processes fighting per CPU
               // = constant context switching = slow = timeouts = 64% success
```

```
What's happening right now:
CPU Core 1:  [Process 1] → [Process 2] → [Process 3] → [Process 1] → ...
CPU Core 2:  [Process 4] → [Process 5] → [Process 6] → [Process 4] → ...
             ↑ 3 Node processes sharing 1 CPU core
             ↑ Each context switch = 1–10 microseconds of wasted time
             ↑ Under load = cascading timeouts
```

Bug 2 — Fake PM2 Config Keys Doing Nothing

```
// YOUR CURRENT CONFIG — these keys DO NOT EXIST in PM2
load_balancing: {
  strategy: "round-robin",          // ← NOT a PM2 option, silently ignored
  max_requests_per_instance: 1000,  // ← NOT a PM2 option, silently ignored
  max_concurrent_requests: 50,      // ← NOT a PM2 option, silently ignored
},
```

PM2 ignores unknown keys completely. You have zero request limiting in place.

Fix Right Now — 5 Minutes

```
// ecosystem.config.js — REPLACE ENTIRELY WITH THIS
module.exports = {
  apps: [
    {
      name: "rupeeflow-api",
      script: "./dist/src/main.js",

      // m6a.large = 2 vCPU. For I/O-heavy NestJS, use 2x vCPU = 4.
```

```
      // Do NOT use "max" — that would still give 6 on m6a.large (2 physical +
hyperthreading)
      instances: 4,
      exec_mode: "cluster",

      // m6a.large has 8GB RAM. 4 processes × 1.5GB each = 6GB.
      // Leaves 2GB for Redis (Docker) and OS. Correct.
      max_memory_restart: "1500M",

      // These ARE valid PM2 options
      kill_timeout: 10000,        // 10s for graceful shutdown (fixes A-8 from
audit)
      wait_ready: true,           // wait for app to emit 'ready' signal before
routing
      listen_timeout: 20000,      // 20s for app to become ready

      env: {
        NODE_ENV: "production",
        TZ: "Asia/Kolkata",       // "Calcutta" is deprecated, use "Kolkata"
      },

      // Logs
      error_file: "./logs/err.log",
      out_file: "./logs/out.log",
      merge_logs: true,
      log_date_format: "YYYY-MM-DD HH:mm:ss.SSS",
      log_file: "./logs/combined.log",

      // Stability
      min_uptime: "30s",
      max_restarts: 5,            // 5 restarts then PM2 stops trying (prevent
restart loops)
      restart_delay: 3000,
      watch: false,

      // Graceful reload (zero-downtime deploy)
      // When you run: pm2 reload rupeeflow-api
      // PM2 starts new processes, waits for 'ready', then kills old ones
    },
  ],
};
```

Deploy this fix now:

```
# On your EC2:
cd /your/app/directory
# Edit ecosystem.config.js with above content, then:
pm2 reload ecosystem.config.js --update-env
pm2 save
```

Add this to `src/main.ts` to send the 'ready' signal:

```
async function bootstrap() {
  // ... your existing setup ...
  await app.listen(port);

  // Tell PM2 this process is ready to receive traffic
  if (process.send) {
    process.send("ready");
  }

  Logger.log(`Rupeeflow API running on port ${port}`, "Bootstrap");
}
```

Part 1 — Understanding Your Current State

What You Have Today

```
                    Internet
                        |
                        | (direct to EC2, no load balancer)
                        ▼
              ┌──────────────────────┐
              |   EC2 m6a.large      |   2 vCPU, 8GB RAM — ~$73/month
              |                      |
              |  NestJS (PM2 × 6)    |   ← Bug: should be × 4
              |  Redis (Docker)      |   ← Risk: same machine as app
              |  Nginx               |
              |  PgBouncer?          |   ← Missing (causing DB exhaustion)
              └──────────┬───────────┘
                         | Private VPC
                         ▼
              ┌──────────────────────┐
              |   RDS PostgreSQL     |   Multi-AZ — ~$300-500/month
              |   (Multi-AZ)         |   ← Overkill, paying 2x
              └──────────────────────┘
```

What's Breaking and Why

```
PROBLEM                          ROOT CAUSE                  IMPACT
──────────────────────────────────────────────────────────────────────

64% API success rate             6 PM2 on 2 vCPU             Timeouts
                                 → CPU contention

DB connection errors             No PgBouncer                500 errors
"too many connections"           → pool exhausted             under load

Redis = SPOF                     Redis on app server          Queue jobs lost
                                 → EC2 restart = Redis gone   on restart

No zero-downtime deploy          No load balancer             Downtime on
```

```
                                    → PM2 reload drops           every deploy
                                      requests during reload

$900/month                          RDS Multi-AZ + NAT Gateway   Burning money
                                    → see AWS Cost PDF
```

Part 2 — The 4-Phase Infrastructure Plan

We go from single fragile EC2 to production-grade in 4 phases. **Each phase works independently — you can stop at any phase.**

```
PHASE 0   Fix existing EC2 (TODAY)           Cost: $0 extra, fixes 64% issue
PHASE 1   Stabilize infrastructure           Cost: ~$280/month (saves $620)
PHASE 2   Add resilience + read replica      Cost: ~$420/month
PHASE 3   Horizontal scaling                 Cost: ~$580/month
PHASE 4   Full production (3Cr/day ready)    Cost: ~$750/month
```

Phase 0 — Fix The Existing Server (Do This Today)

0.1 — Verify Your EC2 Instance Details

```
# SSH into EC2 and run:
nproc                   # shows actual CPU count — should show 2 for m6a.large
free -h                 # shows RAM — should show ~7.5G for m6a.large
df -h                   # shows disk usage

# Check what's using memory
pm2 list                # see all processes and their memory
pm2 monit               # live monitoring of CPU and memory per process

# Check if Redis is running
docker ps               # see running containers

# Check DB connections right now
psql -h YOUR_RDS_HOST -U YOUR_USER -c "
  SELECT count(*), state
  FROM pg_stat_activity
  WHERE datname = 'your_db_name'
  GROUP BY state;
"
```

0.2 — Add PgBouncer via Docker (Fixes DB Connection Exhaustion)

Add PgBouncer to your existing `docker-compose.yml` :

```
# docker-compose.yml — your existing file, add this service
version: "3.8"

services:
```

```yaml
  redis:
    image: redis:7.2.4          # pin version, fix from audit DEV-4
    restart: unless-stopped
    command: >
      redis-server
      --maxmemory 1500mb
      --maxmemory-policy allkeys-lru
      --appendonly yes
      --appendfsync everysec
      --save 900 1
      --save 300 10
    ports:
      - "127.0.0.1:6379:6379"   # bind to localhost only — not 0.0.0.0
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 3

  pgbouncer:
    image: bitnami/pgbouncer:1.22.1
    restart: unless-stopped
    environment:
      POSTGRESQL_HOST: ${DB_HOST}           # your RDS hostname
      POSTGRESQL_PORT: 5432
      POSTGRESQL_USERNAME: ${DB_USERNAME}
      POSTGRESQL_PASSWORD: ${DB_PASSWORD}
      POSTGRESQL_DATABASE: ${DB_NAME}
      PGBOUNCER_POOL_MODE: transaction
      PGBOUNCER_MAX_CLIENT_CONN: 400        # up to 400 app connections
      PGBOUNCER_DEFAULT_POOL_SIZE: 20       # 20 real DB connections (RDS limit
aware)
      PGBOUNCER_MIN_POOL_SIZE: 5
      PGBOUNCER_RESERVE_POOL_SIZE: 5
      PGBOUNCER_SERVER_IDLE_TIMEOUT: 600
      PGBOUNCER_CLIENT_IDLE_TIMEOUT: 60
      PGBOUNCER_AUTH_TYPE: scram-sha-256
    ports:
      - "127.0.0.1:6432:6432"   # localhost only
    healthcheck:
      test: ["CMD", "pg_isready", "-h", "localhost", "-p", "6432"]
      interval: 10s
      timeout: 5s
      retries: 3

volumes:
  redis-data:
```

Update your `src/config/db.config.ts` to connect through PgBouncer:

```
export const dbConfig: TypeOrmModuleOptions = {
  type: "postgres",
  host: "127.0.0.1",      // PgBouncer on localhost
  port: 6432,             // PgBouncer port
  username,
  password,
  database: name,
  entities: [__dirname + "/../**/*.entity{.ts,.js}"],
  synchronize: false,
  maxQueryExecutionTime: 2000,
  ssl: false,
  extra: {
    max: 25,              // 4 PM2 processes × 25 = 100 connections to PgBouncer
    min: 3,
    connectionTimeoutMillis: 10000,
    idleTimeoutMillis: 30000,
    statement_timeout: 8000,
    lock_timeout: 3000,
    idle_in_transaction_session_timeout: 5000,
    prepareThreshold: 0,  // MANDATORY with PgBouncer transaction mode
  },
};
```

0.3 — Nginx Config (If Not Already Correct)

```
# Check current Nginx config
sudo nginx -t
cat /etc/nginx/sites-enabled/rupeeflow

# If Nginx not installed:
sudo apt update && sudo apt install -y nginx
```

Correct `/etc/nginx/sites-available/rupeeflow`:

```
# Rate limiting zone — 50 req/sec per IP at Nginx level
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=50r/s;
limit_req_zone $binary_remote_addr zone=otp_limit:10m  rate=3r/m;

server {
    listen 80;
    server_name api.rupeeflow.in;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.rupeeflow.in;

    ssl_certificate     /etc/letsencrypt/live/api.rupeeflow.in/fullchain.pem;
```

```
    ssl_certificate_key /etc/letsencrypt/live/api.rupeeflow.in/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_session_cache   shared:SSL:10m;
    ssl_session_timeout 10m;

    # Fix S-3 from audit: trust only immediate proxy header
    # (Nginx is the first proxy, so use $remote_addr for real IP)
    # If behind AWS ALB later, add: real_ip_header X-Forwarded-For;

    # Security headers (supplement Helmet in NestJS)
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;
    add_header X-Content-Type-Options nosniff always;

    # Request size limit (match NestJS)
    client_max_body_size 10m;

    # Timeouts
    proxy_connect_timeout 10s;
    proxy_send_timeout    30s;
    proxy_read_timeout    30s;

    # Default: rate limited
    location / {
        limit_req zone=api_limit burst=50 nodelay;

        proxy_pass         http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $remote_addr;  # use $remote_addr not
$proxy_add_x_forwarded_for
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_set_header   Connection        "";
    }

    # OTP endpoints: stricter rate limit
    location ~ ^/api/v1/auth/(send-signup-otp|send-forgot-password-otp) {
        limit_req zone=otp_limit burst=2 nodelay;

        proxy_pass         http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $remote_addr;
        proxy_set_header   X-Forwarded-Proto $scheme;
    }

    # Webhook endpoints: no rate limit (PGs may send bursts)
    location ~ ^/api/v1/payments/.*/webhook {
        limit_req off;
```

```
        proxy_pass         http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $remote_addr;
        proxy_set_header   X-Forwarded-Proto $scheme;
    }

    # Block dangerous paths
    location ~* \.(env|git|htaccess|sql|bak)$ { return 404; }
    location = /favicon.ico { return 204; access_log off; }
}
```

```
# Apply Nginx config
sudo ln -s /etc/nginx/sites-available/rupeeflow /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

0.4 — Server Hardening (One-Time)

```
# 1. Disable root login over SSH
sudo sed -i 's/PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config
sudo systemctl restart sshd

# 2. Set up automatic security updates
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades

# 3. Install fail2ban (blocks IPs after failed SSH attempts)
sudo apt install -y fail2ban
sudo systemctl enable fail2ban && sudo systemctl start fail2ban

# 4. Install logrotate for PM2 logs
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 50M
pm2 set pm2-logrotate:retain 7
pm2 set pm2-logrotate:compress true

# 5. Increase file descriptor limits for Node.js
echo "rupeeflow soft nofile 65536" | sudo tee -a /etc/security/limits.conf
echo "rupeeflow hard nofile 65536" | sudo tee -a /etc/security/limits.conf
# Add to ecosystem.config.js: node_args: "--max-old-space-size=1024"
```

Phase 1 — Stabilize (Week 1–2) — $280/month

Apply all cost optimizations from the AWS Cost PDF first. After Phase 0 + Cost PDF + Phase 1, you should have:

```
Internet
    | (HTTPS only)
```

```
    ▼
EC2 m6a.large
├── Nginx (SSL termination + rate limiting)
├── NestJS (PM2 × 4) — correct for 2 vCPU
├── PgBouncer Docker — connection pooling
└── Redis Docker — capped at 1.5GB

    ↓ private VPC (no NAT Gateway)

RDS PostgreSQL Single-AZ (after Multi-AZ removal)
```

Expected result after Phase 0 + 1:

- Success rate: 64% → 95%+
- Monthly cost: $900 → ~$280

Phase 2 — Add Resilience (Week 3–6) — $420/month

2.1 — Upgrade EC2 Instance

m6a.large (2 vCPU, 8GB) is too small for production fintech. At 347 average TPS, each vCPU needs to handle 87 requests/second — too much when also running Redis and PgBouncer.

Upgrade path:

Instance	vCPU	RAM	Cost/mo (reserved)	PM2 instances	Capacity
m6a.large	2	8GB	~$44	4	~200 TPS
m6a.xlarge	**4**	**16GB**	**~$86**	**8**	**~600 TPS**
m6a.2xlarge	8	32GB	~$168	16	~1,200 TPS

Upgrade to m6a.xlarge now. Cost difference: +$42/month. Capacity gain: 3x.

```
AWS Console → EC2 → Your instance
→ Instance state → Stop
→ Actions → Instance settings → Change instance type
→ Select: m6a.xlarge
→ Start instance
```

Update ecosystem.config.js after upgrade:

```
instances: 8,            // 4 vCPU × 2 (I/O bound)
max_memory_restart: "1500M",  // 8 × 1.5GB = 12GB of 16GB — Redis + OS get 4GB
```

2.2 — Add RDS Read Replica (Analytics on Separate Server)

This moves all your dashboard analytics queries (business trends, conversion rates, hourly stats) to a dedicated read server, freeing your primary RDS for live payment writes.

```
AWS Console → RDS → Your DB → Actions → Create read replica
→ DB instance identifier: rupeeflow-db-replica
```

```
→ Instance type: db.t3.medium (analytics queries don't need huge instance)
→ Multi-AZ: No
→ Storage type: gp3
→ Publicly accessible: No
→ Create
```

Add to `.env`:

```
DB_READ_HOST=rupeeflow-db-replica.xxxx.ap-south-1.rds.amazonaws.com
```

Update TypeORM config for read/write split:

```
// src/config/db.config.ts
export const dbConfig: TypeOrmModuleOptions = {
  type: "postgres",
  replication: {
    master: {
      host: "127.0.0.1",   // PgBouncer (points to primary RDS)
      port: 6432,
      username, password, database: name,
    },
    slaves: [{
      host: process.env.DB_READ_HOST,   // direct to read replica
      port: 5432,
      username, password, database: name,
    }],
  },
  entities: [__dirname + "/../**/*.entity{.ts,.js}"],
  synchronize: false,
  extra: { max: 20, min: 3, prepareThreshold: 0 },
};
```

TypeORM automatically sends all `find()`, `findOne()`, SELECT queries to the replica and writes to master.

2.3 — Application Load Balancer (Zero-Downtime Deploys)

Even with a single EC2, adding an ALB gives you:

- Health checks — ALB stops routing if your app crashes
- Zero-downtime deploys — drain connections before swapping
- Easy path to add a second EC2 later (just register it with the same ALB)

```
AWS Console → EC2 → Load Balancers → Create load balancer
→ Application Load Balancer
→ Name: rupeeflow-alb
→ Scheme: Internet-facing
→ Listeners: HTTP:80 (redirect to HTTPS), HTTPS:443
→ Availability Zones: ap-south-1a, ap-south-1b (minimum 2 AZs required)
→ Security group: new SG, allow 80+443 from anywhere
```

```
→ Target Group:
   Name: rupeeflow-targets
   Target type: Instance
   Protocol: HTTP, Port: 3000  (direct to NestJS, bypass Nginx if using ALB)
   Health check: /api/health-check
   Healthy threshold: 2
   Unhealthy threshold: 3
   Timeout: 5s
   Interval: 15s

→ Register your EC2 instance as target
```

Update Route 53 DNS:

```
Remove:  api.rupeeflow.in → EC2 public IP  (A record)
Add:     api.rupeeflow.in → ALB DNS name   (CNAME or Alias record)
```

Cost of ALB: ~$18/month + $0.008/LCU. For your traffic, total ~$20-25/month.

2.4 — Zero-Downtime Deployment Script

```bash
#!/bin/bash
# deploy.sh — run this on your EC2 for every deployment

set -e  # exit on any error

APP_DIR="/home/ubuntu/rupeeflow-api"
LOG_FILE="/home/ubuntu/deploy.log"

echo "$(date): Starting deployment" >> $LOG_FILE

cd $APP_DIR

# Pull latest code
git pull origin main 2>&1 | tee -a $LOG_FILE

# Install dependencies (only if changed)
pnpm install --frozen-lockfile 2>&1 | tee -a $LOG_FILE

# Build
pnpm build 2>&1 | tee -a $LOG_FILE

# Run DB migrations
pnpm mig:run 2>&1 | tee -a $LOG_FILE

# Graceful reload — PM2 starts new processes, waits for 'ready' signal,
# then stops old ones. Zero dropped requests.
pm2 reload ecosystem.config.js --update-env 2>&1 | tee -a $LOG_FILE

echo "$(date): Deployment complete" >> $LOG_FILE
```

```
# Verify health
sleep 5
curl -sf http://localhost:3000/api/health-check && \
  echo "$(date): Health check passed" >> $LOG_FILE || \
  (echo "$(date): Health check FAILED — rolling back" >> $LOG_FILE && pm2 revert
rupeeflow-api)
```

Phase 3 — Horizontal Scaling (Month 2–3) — $580/month

3.1 — Move Redis to ElastiCache

Why: Redis on your EC2 means:

- If you add a second EC2, they can't share Redis (each has its own)
- If EC2 reboots, all queued Bull jobs are lost
- Redis memory competes with your Node processes

Move to ElastiCache for Redis:

```
AWS Console → ElastiCache → Create cluster
→ Cluster mode: Disabled (single node for now, easy to enable cluster later)
→ Engine version: Redis 7.2
→ Node type: cache.t3.medium (1.5GB RAM, ~$25/month)
→ Number of replicas: 1 (for read availability)
→ Subnet group: same VPC as your EC2
→ Security group: new SG, allow 6379 from EC2 security group only
```

After ElastiCache is up:

1. Update `.env` : `REDIS_HOST=your-elasticache-endpoint.cache.amazonaws.com`
2. Remove Redis from `docker-compose.yml`
3. Update `docker-compose.yml` for PgBouncer only

3.2 — Add Second EC2 + Auto Scaling Group

```
AWS Console → EC2 → Launch Templates → Create launch template
→ Name: rupeeflow-app-template
→ AMI: Create AMI from your current EC2 first (snapshot)
→ Instance type: m6a.xlarge
→ Key pair: your existing key
→ Security group: your app security group
→ User data (runs on startup):
```

```
#!/bin/bash
# User data script — runs when new EC2 starts
cd /home/ubuntu/rupeeflow-api
git pull origin main
pnpm install --frozen-lockfile
pnpm build
pm2 start ecosystem.config.js
```

```
pm2 save
pm2 startup
```

```
AWS Console → EC2 → Auto Scaling Groups → Create
→ Launch template: rupeeflow-app-template
→ VPC: your VPC
→ Availability Zones: ap-south-1a, ap-south-1b
→ Load balancer: attach rupeeflow-alb target group
→ Health check: ELB (use ALB health checks)
→ Desired: 2, Min: 2, Max: 4

Scaling policies:
→ Scale out: CPU > 70% for 2 consecutive 3-min periods → add 1 instance
→ Scale in:  CPU < 30% for 5 consecutive 3-min periods → remove 1 instance
```

Architecture after Phase 3:

```
Internet
    │
    ▼
AWS ALB
├── EC2 m6a.xlarge #1 (PM2 × 8, NestJS)
└── EC2 m6a.xlarge #2 (PM2 × 8, NestJS)
         │
    ┌────┴──────────────────────┐
    │                           │
ElastiCache Redis        PgBouncer (on each EC2)
(single cluster)              │
                              ▼
                     RDS PostgreSQL Primary (writes)
                     RDS PostgreSQL Replica (reads/analytics)
```

Phase 4 — Production Scale for 3 Crore/Day (Month 4+) — $750/month

4.1 — What 3 Crore/Day Actually Needs

```
Required:                    Phase 3 Capacity:    Gap?
──────────────────────────────────────────────────────
1,050 peak TPS               ~1,200 TPS           ✓ covered
4 EC2 m6a.xlarge max         Auto scaling 2→4     ✓ covered
5,000 DB ops/sec             PgBouncer + replica  ✓ covered
Redis < 50ms                 ElastiCache          ✓ covered
Zero lost payments           DLQ + idempotency    ← code fix needed
99.9% uptime                 Multi-AZ RDS *        ← re-enable at this phase
```

At Phase 4 scale, bring Multi-AZ back — now you have an ALB + 2+ EC2s to justify it.

4.2 — Enable RDS Multi-AZ Again (At Phase 4)

At Phase 4 you have 2+ EC2 instances. Now Multi-AZ makes sense because:

- If primary RDS fails, 60-second failover doesn't take down your app
- Your EC2s reconnect to the new primary automatically

```
AWS Console → RDS → Your instance → Modify
→ Multi-AZ: Yes
→ Apply: During next maintenance window
```

4.3 — AWS WAF (Block Attacks at Edge)

At 1,000+ TPS, you will get attacked. WAF blocks before traffic hits your EC2.

```
AWS Console → WAF & Shield → Create web ACL
→ Name: rupeeflow-waf
→ Resource: your ALB
→ Add managed rule groups:
  ✓ AWS-AWSManagedRulesCommonRuleSet    (OWASP top 10)
  ✓ AWS-AWSManagedRulesKnownBadInputs  (SQL injection, XSS)
  ✓ AWS-AWSManagedRulesBotControlRuleSet (bot traffic)
→ Add custom rules:
  Rule: Rate limit per IP
  → If IP sends > 300 requests in 5 minutes → Block
```

Cost: WAF = ~$5/month base + $0.60/million requests = ~$15-25/month at your scale.

4.4 — CloudFront for Static Assets

Your payment pages (GeoPay checkout, payment links) currently served from EC2. Move them to CloudFront:

- Faster load times globally
- Reduces EC2 load by ~15-20%
- S3 + CloudFront = ~$5/month

4.5 — Monitoring Dashboard (CloudWatch)

```
# Install CloudWatch agent on each EC2
wget https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
sudo dpkg -i amazon-cloudwatch-agent.deb

# Config: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
```

```
{
  "metrics": {
    "namespace": "Rupeeflow/App",
    "metrics_collected": {
      "mem": {
        "measurement": ["mem_used_percent"],
        "metrics_collection_interval": 60
      },
      "disk": {
```

```
        "measurement": ["disk_used_percent"],
        "resources": ["/"],
        "metrics_collection_interval": 300
      },
      "net": {
        "measurement": ["net_bytes_sent", "net_bytes_recv"],
        "metrics_collection_interval": 60
      }
    }
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/home/ubuntu/rupeeflow-api/logs/err.log",
            "log_group_name": "/rupeeflow/app/errors",
            "log_stream_name": "{instance_id}",
            "retention_in_days": 30
          },
          {
            "file_path": "/home/ubuntu/rupeeflow-api/logs/combined.log",
            "log_group_name": "/rupeeflow/app/combined",
            "log_stream_name": "{instance_id}",
            "retention_in_days": 7
          }
        ]
      }
    }
  }
}
```

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 \
  -s -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
```

Part 3 — CI/CD Pipeline (Stop SSHing to Deploy)

Every time you SSH into production to deploy, you risk human error. Set up GitHub Actions.

GitHub Actions Workflow

```
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]
```

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "20"

      - name: Install pnpm
        uses: pnpm/action-setup@v2
        with:
          version: latest

      - name: Install dependencies
        run: pnpm install --frozen-lockfile

      - name: Lint
        run: pnpm lint

      - name: Build
        run: pnpm build

      - name: Deploy to EC2
        uses: appleboy/ssh-action@v1.0.0
        with:
          host: ${{ secrets.EC2_HOST }}        # store in GitHub Secrets
          username: ubuntu
          key: ${{ secrets.EC2_SSH_KEY }}
          script: |
            cd /home/ubuntu/rupeeflow-api
            git pull origin main
            pnpm install --frozen-lockfile
            pnpm build
            pnpm mig:run
            pm2 reload ecosystem.config.js --update-env
            sleep 5
            curl -sf http://localhost:3000/api/health-check || exit 1
            echo "Deploy successful"

      - name: Notify on failure
        if: failure()
        run: |
          # Send Slack/email alert on failed deployment
          echo "Deployment failed on $(date)"
```

GitHub Secrets to add:

```
EC2_HOST       = your EC2 public IP or domain
EC2_SSH_KEY    = your PEM file content (cat your-key.pem)
```

Part 4 — Cost Summary by Phase

```
                              MONTHLY COST    CAPACITY      SUCCESS RATE
──────────────────────────────────────────────────────────────────────────
Current state                 ~$900/month     45 req/sec    64%
──────────────────────────────────────────────────────────────────────────
Phase 0 (PM2 fix, free)        $900/month     200+ req/sec  90%+
Phase 0 + Cost PDF fixes       ~$280/month    200+ req/sec  90%+
Phase 1 (Stabilize)            ~$280/month    300 req/sec   95%+
Phase 2 (m6a.xlarge + ALB + replica) ~$420   600 req/sec   98%+
Phase 3 (2 EC2 + ElastiCache)  ~$580/month   1,200 req/sec 99%+
Phase 4 (4 EC2 + WAF + CDN)    ~$750/month   3,000 req/sec 99.9%+
──────────────────────────────────────────────────────────────────────────
3 Crore/day needs: 1,050 peak TPS → achievable at Phase 3 ($580/month)
```

Part 5 — Checklist: What to Do and When

This Week (Phase 0 — Zero Cost)

```
☐ Fix ecosystem.config.js: instances 6 → 4, remove fake keys
☐ Add process.send('ready') to main.ts
☐ Start PgBouncer via docker-compose
☐ Update db.config.ts to connect via 127.0.0.1:6432
☐ Fix Nginx config (rate limiting + correct X-Forwarded-For)
☐ Set Redis maxmemory 1500mb in docker-compose
☐ Run: pm2 reload ecosystem.config.js --update-env
☐ Monitor: pm2 monit — verify success rate improves
```

Week 2 (Cost Savings)

```
☐ Switch RDS Multi-AZ → Single-AZ (2 min downtime, saves ~$300/mo)
☐ Switch RDS storage gp2 → gp3 (no downtime, saves ~$20/mo)
☐ Delete NAT Gateway if not needed (saves ~$100/mo)
☐ Set CloudWatch log retention to 30 days
☐ Set up billing alert at $300 and $400
☐ Buy EC2 reserved instance (1-year)
```

Week 3–4 (Phase 2)

```
☐ Upgrade EC2: m6a.large → m6a.xlarge
☐ Update PM2 instances from 4 → 8
☐ Create RDS read replica
☐ Set up Application Load Balancer
```

```
☐ Update DNS to point to ALB
☐ Set up GitHub Actions CI/CD
```

Month 2 (Phase 3)

```
☐ Create ElastiCache Redis (cache.t3.medium)
☐ Remove Redis from docker-compose
☐ Create EC2 Launch Template (AMI from current instance)
☐ Create Auto Scaling Group (min 2, max 4)
☐ Verify both EC2s share same ElastiCache and RDS
☐ Test: stop one EC2 — verify traffic routes to second automatically
```

Month 4 (Phase 4 — 3Cr/day ready)

```
☐ Apply all code fixes from SCALE_TO_3CR.pdf
☐ Re-enable RDS Multi-AZ
☐ Add AWS WAF to ALB
☐ Set up CloudFront for static assets
☐ Set up CloudWatch Dashboard
☐ Load test: verify 1,050 TPS with 99%+ success rate
```

Quick Reference — Instance Sizing Guide

```
m6a.large  (2 vCPU,  8GB)  → PM2 × 4  → ~300 TPS  → fine for now
m6a.xlarge (4 vCPU, 16GB)  → PM2 × 8  → ~600 TPS  → upgrade to this
m6a.2xlarge(8 vCPU, 32GB)  → PM2 × 16 → ~1,200 TPS → only if needed

Rule: PM2 instances = vCPU × 2 (NestJS is I/O-heavy)
Rule: max_memory_restart = (total RAM - Redis RAM - OS RAM) / PM2 instances
      m6a.xlarge: (16GB - 2GB Redis - 2GB OS) / 8 = 1.5GB per process ✓
```

End of Document — Rupeeflow Infrastructure Scale Guide — March 2026 Phase 0 alone (free, 30 minutes) should fix your 64% success rate.